COMP0222 Simultaneous Localization and Mapping
Lecture 12: Quantifying the Performance of SLAM Systems

Simon Julier

s.julier@ucl.ac.uk

Motivation

- We've seen the basic theory of how SLAM system work
- We've gone through a couple of widely used SLAM / mapping systems in detail
- However, we haven't answered the question: *how well do they work?*

Structure

- Evaluating the Quality of the Joint State Estimate
- Estimating the Quality of the Platform Estimate
- Estimating the Robustness of the Map
- Estimating the Localization Ability of the Map

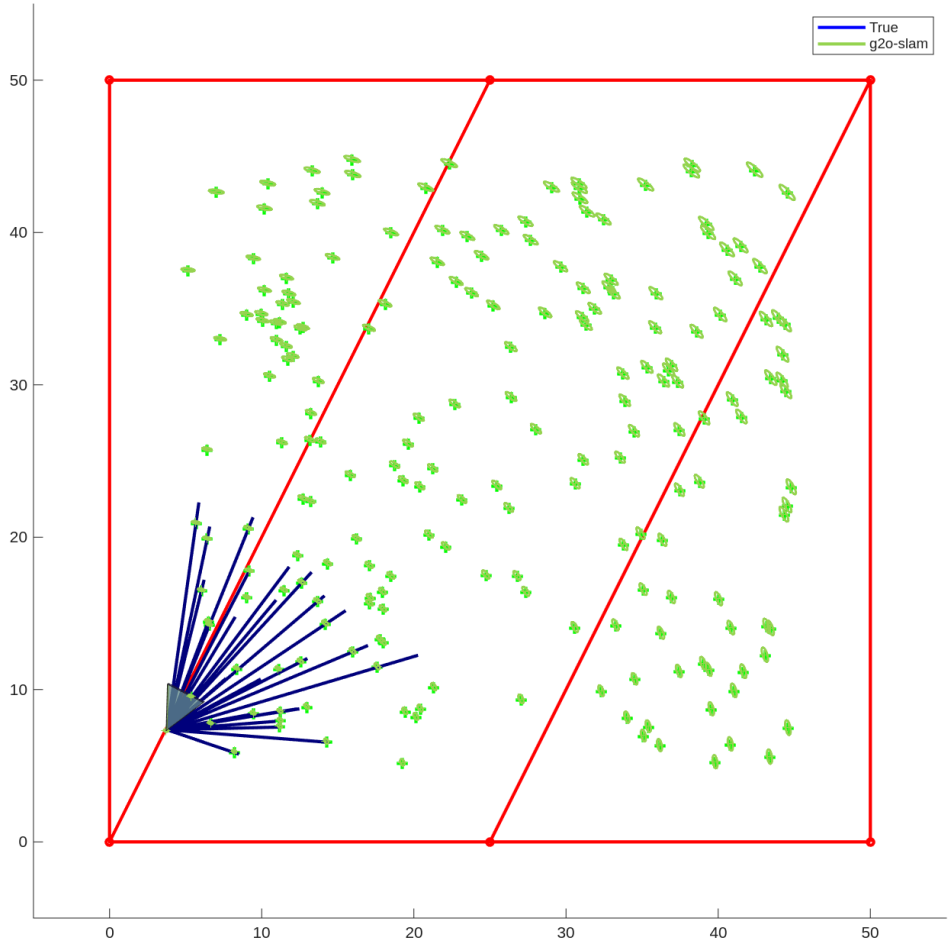
Evaluating the Quality of the Joint State Estimate

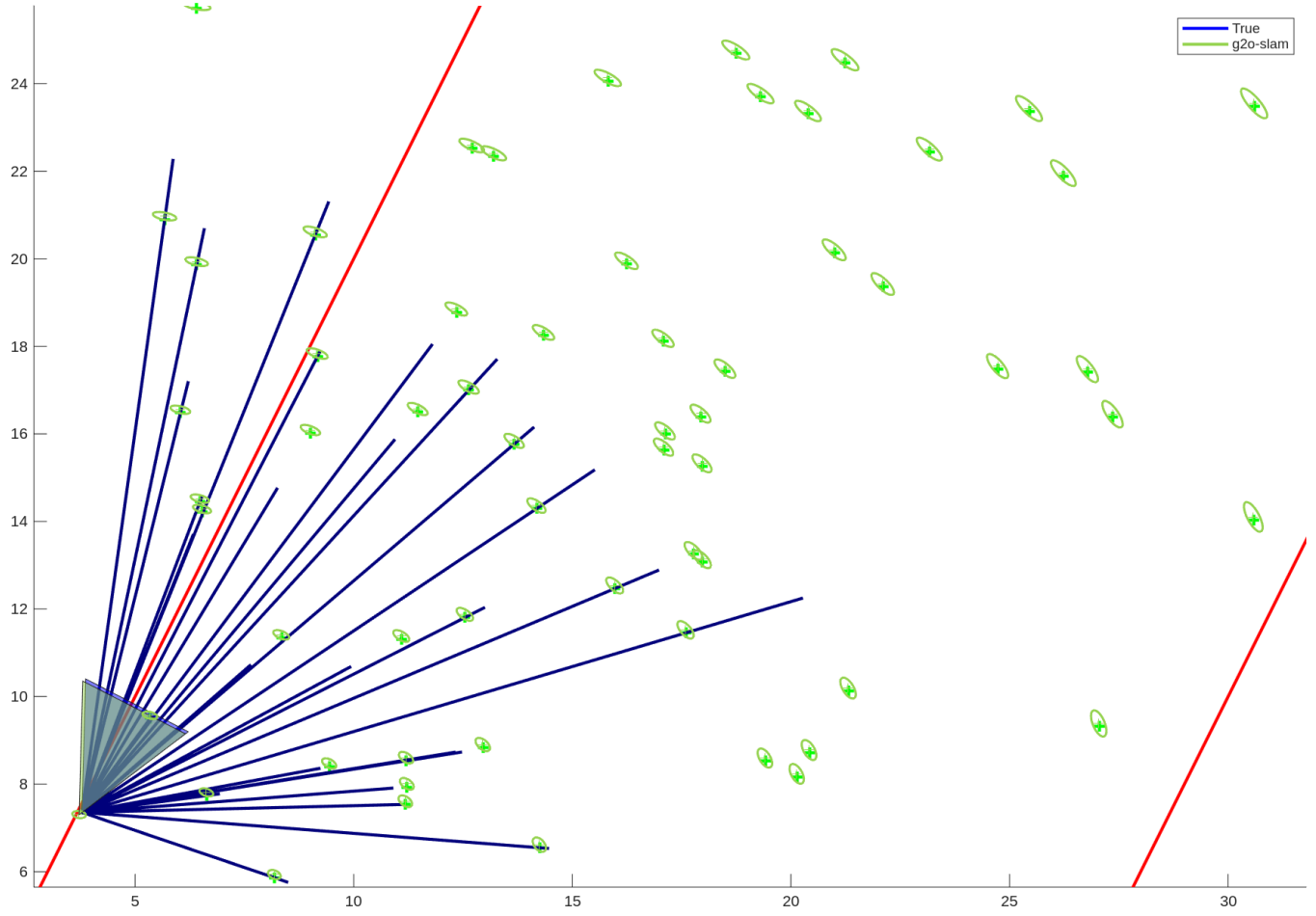
- **Evaluating the Quality of the Joint State Estimate**
- ***Estimating the Quality of the Platform Estimate***
- ***Estimating the Robustness of the Map***
- ***Estimating the Localization Ability of the Map***

Evaluating the Quality of the Joint State Estimate

- Recall that for both for Kalman filters and factor graphs we end up with:
  - An estimate of the landmark locations
  - An estimate of the platform poses over time

- Therefore, can't we just compute the mean squared error in the state estimate and use that as a measure of performance?


True
g2o-slam
50
40
30
20
10
0
0
10
20
30
40
50


True
g2o-slam
24
22
20
18
16
14
12
10
8
6
5
10
15
20
25
30

Issues with Computing the State Error

- ‚There are three challenges we need to handle:
  - We need to have a consistent way to talk about the error in the platform estimate
  - We need to handle the fact that, at any given time, the number of landmark estimates in the SLAM system and in the ground truth are *different*
  - We'd like the result to be a single scalar meaningful number
- We'll see how to do this with SE(2) error and the OSPA metric

Computing the Platform Error

- Recall that the platform state typically contains position and orientation
- For example, in the 2D case,

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \theta_k \end{bmatrix}^\top$$

- The error isn't given by simply taking the difference

SE(2) Pose / Translation Error

- To compute the error we use the *box minus* operator,

$$\tilde{\mathbf{e}}_k = \mathbf{x}_k \boxminus \hat{\mathbf{x}}_k$$

SE(2) Pose / Translation Error

- We first represent the SE(2) estimate as a transformation matrix,

$$\mathbf{x}_k \equiv \mathbf{R}\left[\mathbf{x}_k\right] = \begin{bmatrix} \cos\theta_k & -\sin\theta_k & x_k \\ \sin\theta_k & \cos\theta_k & y_k \\ 0 & 0 & 1 \end{bmatrix}$$

SE(2) Pose / Translation Error

- The error is computed from

$$\mathbf{R}\left[\tilde{\mathbf{x}}_k\right] = \mathbf{R}^{-1}\left[\hat{\mathbf{x}}_k\right]\mathbf{R}\left[\mathbf{x}_k\right]$$

$$= \begin{bmatrix} \cos\tilde{\theta}_k & -\sin\tilde{\theta}_k & \tilde{x}_k \\ \sin\tilde{\theta}_k & \cos\tilde{\theta}_k & \tilde{y}_k \\ 0 & 0 & 1 \end{bmatrix}$$

SE(2) Pose / Translation Error

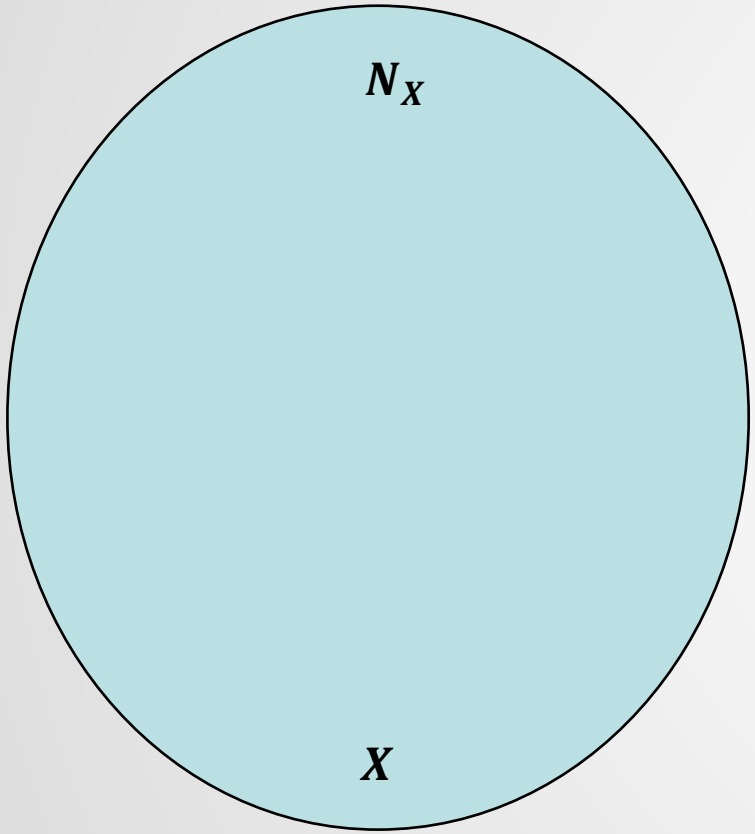
- The translation error is given by

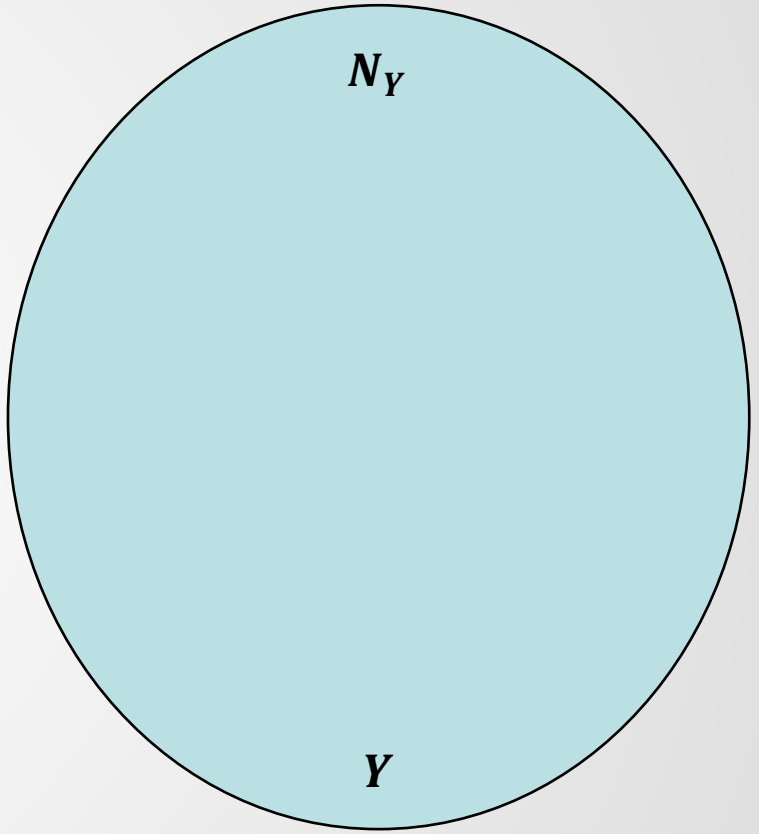
$$\tilde{\mathbf{e}}_k = \begin{bmatrix} \tilde{x}_k \\ \tilde{y}_k \end{bmatrix}$$

Matching Elements in a Set

- It turns out the mathematical model for comparing the landmarks is to think in terms of the difference between two *sets*

Matching Elements in a Set



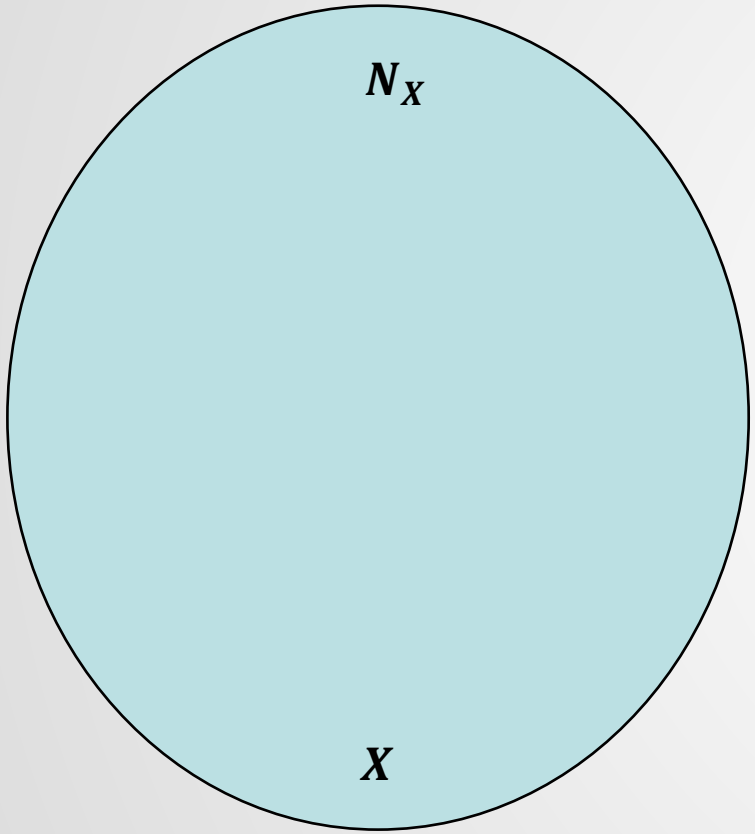


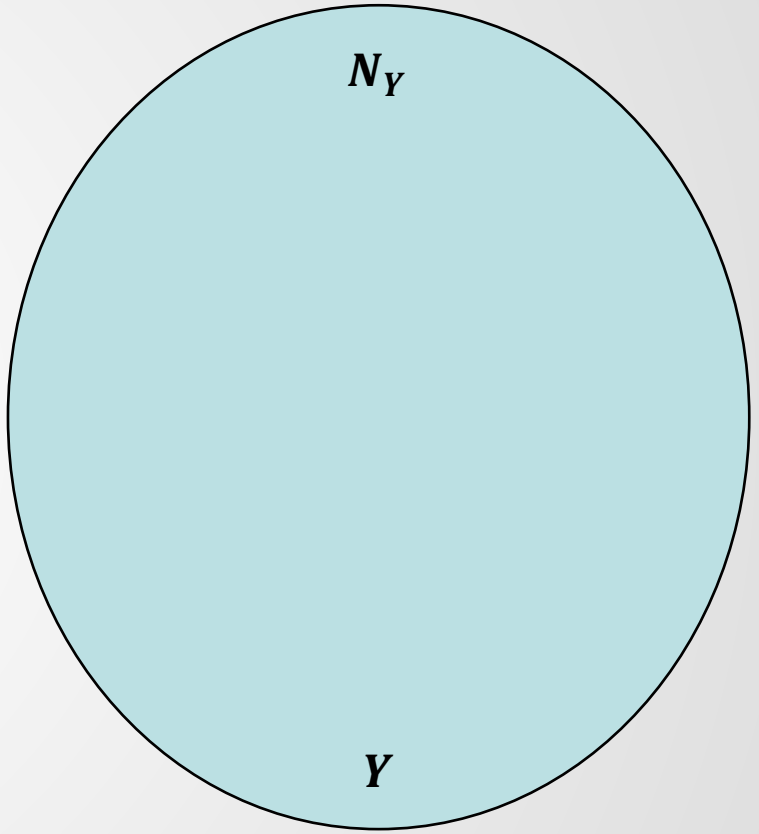
$$N_Y \geq N_X$$

Matching Elements in a Set

- It turns out the mathematical model for comparing the landmarks is to think in terms of the difference between two *sets*
- One common metric is Optimal Sub-pattern Assignment (OSPA)
- It uses *mass transfer* between sets and consist of two parts:
  - Error in the matched elements between the sets
  - Error causes by a difference in cardinality in each set

Matching Elements Between Sets





$$\mathbf{x}_i \Longleftrightarrow \mathbf{y}_{\pi(i)}$$

Thresholded Errors

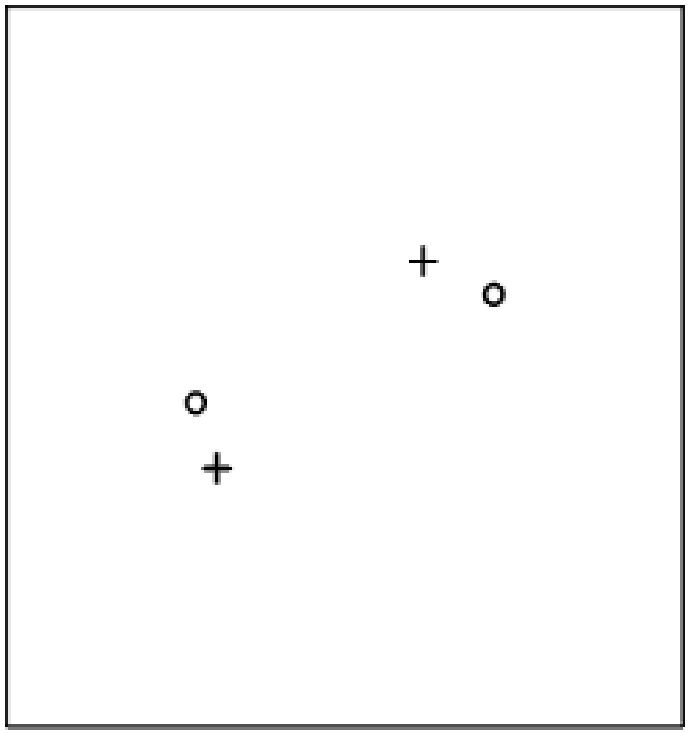
- First consider the error measure,

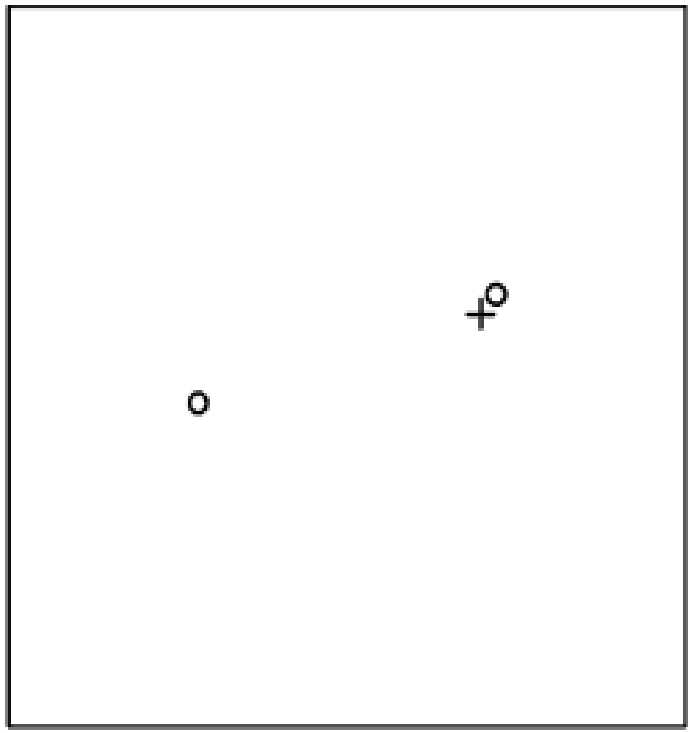
$$d(\mathbf{x}_i, \mathbf{y}_{\pi(i)}) = \left\| \mathbf{x}_i - \mathbf{y}_{\pi(i)} \right\|$$

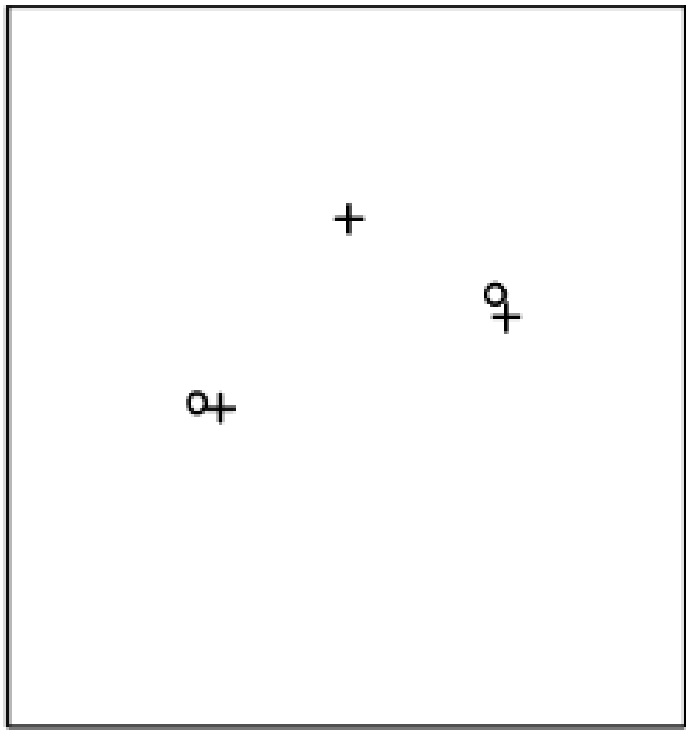
- Introduce the bounded error,

$$d^{(c)}(\mathbf{x}_i, \mathbf{y}_{\pi(i)}) = \min\left(c, \left\| \mathbf{x}_i - \mathbf{y}_{\pi(i)} \right\|\right)$$

Cardinality Errors







"A Consistent Metric for Performance Evaluation of Multi-Object Filters", Schuhmacher, et al., 2008

Cardinality Errors

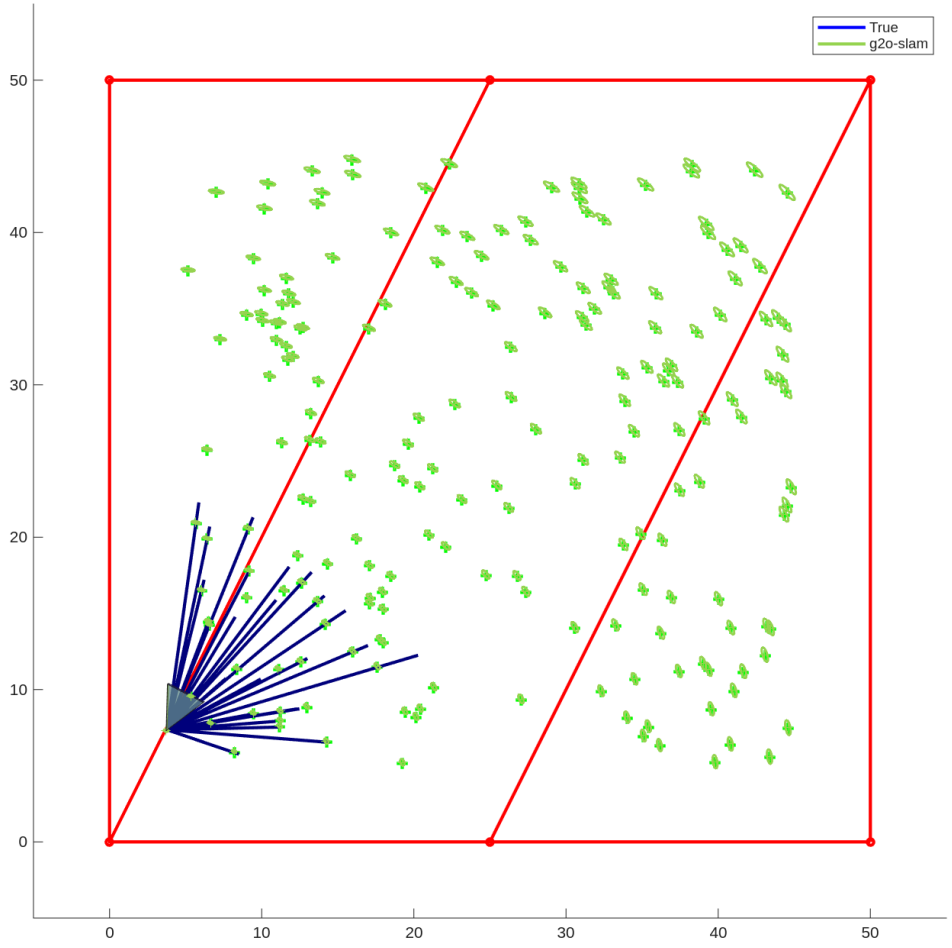
- Cardinality errors are treated like position errors outside of the threshold distance threshold range
- Therefore, the error contribution is

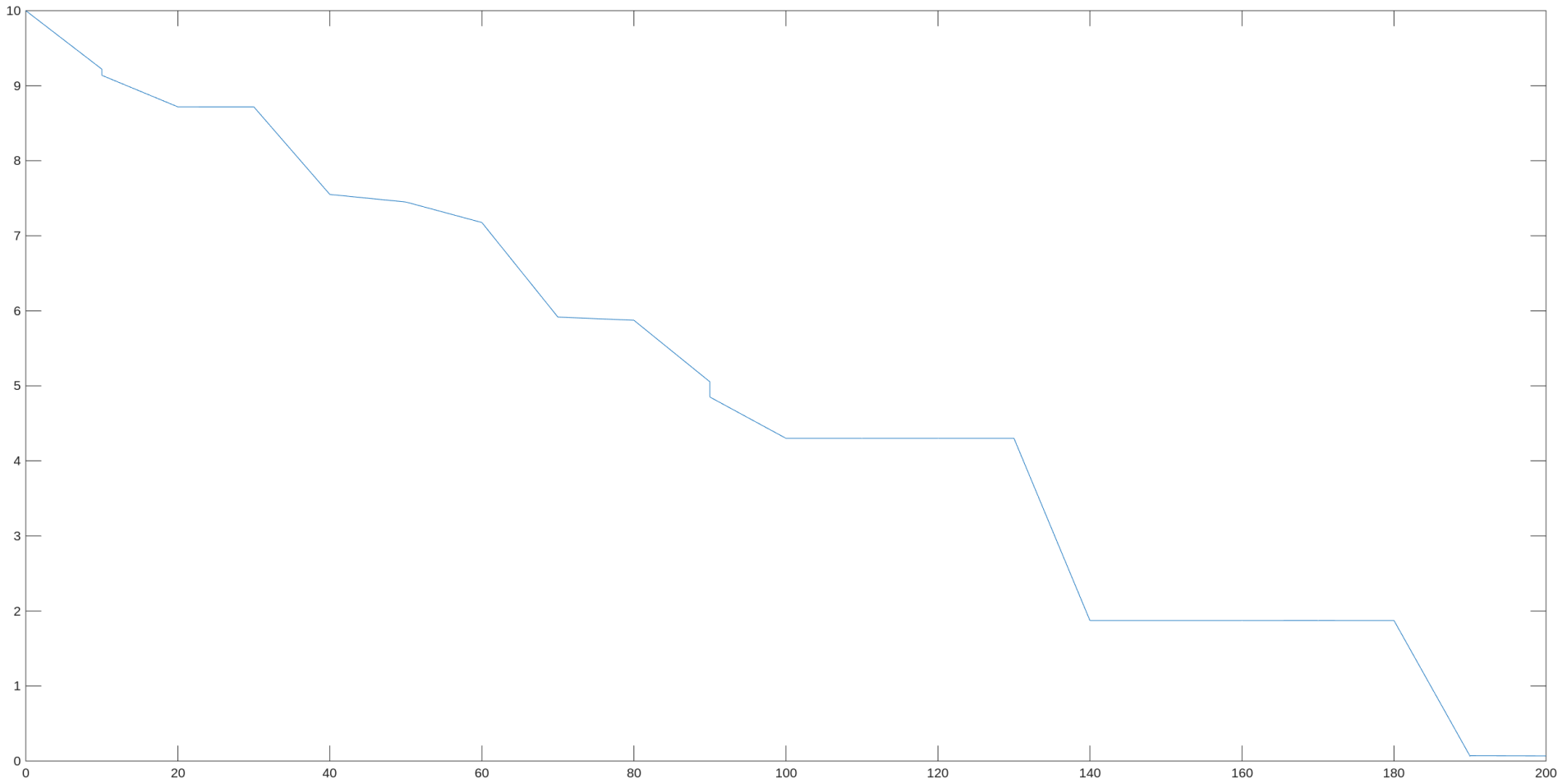
$$c\,(N_Y - N_X)$$

OSPA Metric

- **The full OSPA is a *p*-norm, which computes the average over all the points in the set X**

$$\bar{d}^{(c)}(X,Y) = \left(\frac{1}{N_X} \left(\min_{\pi \in \Pi_n} \sum_{i=1}^{N_X} d^{(c)}\left(\mathbf{x}_i, \mathbf{y}_{\pi(i)}\right)^p + c^p \left(N_Y - N_X\right) \right) \right)^{\frac{1}{p}}$$


True
g2o-slam
50
40
30
20
10
0
0
10
20
30
40
50


10
9
8
7
6
5
4
3
2
1
0
0
20
40
60
80
100
120
140
160
180
200

Practical Limitations with Set-Based Evaluation

- OSPA relies on the fact that we have two clearly identified sets that we want to compare
- The SLAM system provides one set in terms of its landmark estimates
- The other set are the landmarks from the real-world
- However, where do they come from?

Manually Specified Landmarks

Early Lighthouse Prototype. From "Inside Out Tracking"

Naturally Occurring Landmarks

Naturally Occurring Landmarks

What about the Configuration of the Feature Descriptors?

- The feature descriptors depend upon the type of feature detector and the configuration for them
- For example, for ORB we can specify how many features we want to extract per frame
- This can have a *big* impact on the performance of the system

KITTI with 1000, 1500, 2000, 4000 ORB Features per Frame

1000

1500

2000

4000

KITTI with 8000 ORB Features Per Frame

KITTI with 8000 ORB Features Per Frame (Cont'd)

Summary

- We can use error in the state estimate for a SLAM system when we can control the number of detectable landmarks in the environment
- In real environments, we can't do this
- Therefore we have to measure SLAM system performance in a more indirect manner
- We'll start by just assessing how good the platform trajectory is

Estimating the Quality of the Platform Estimate

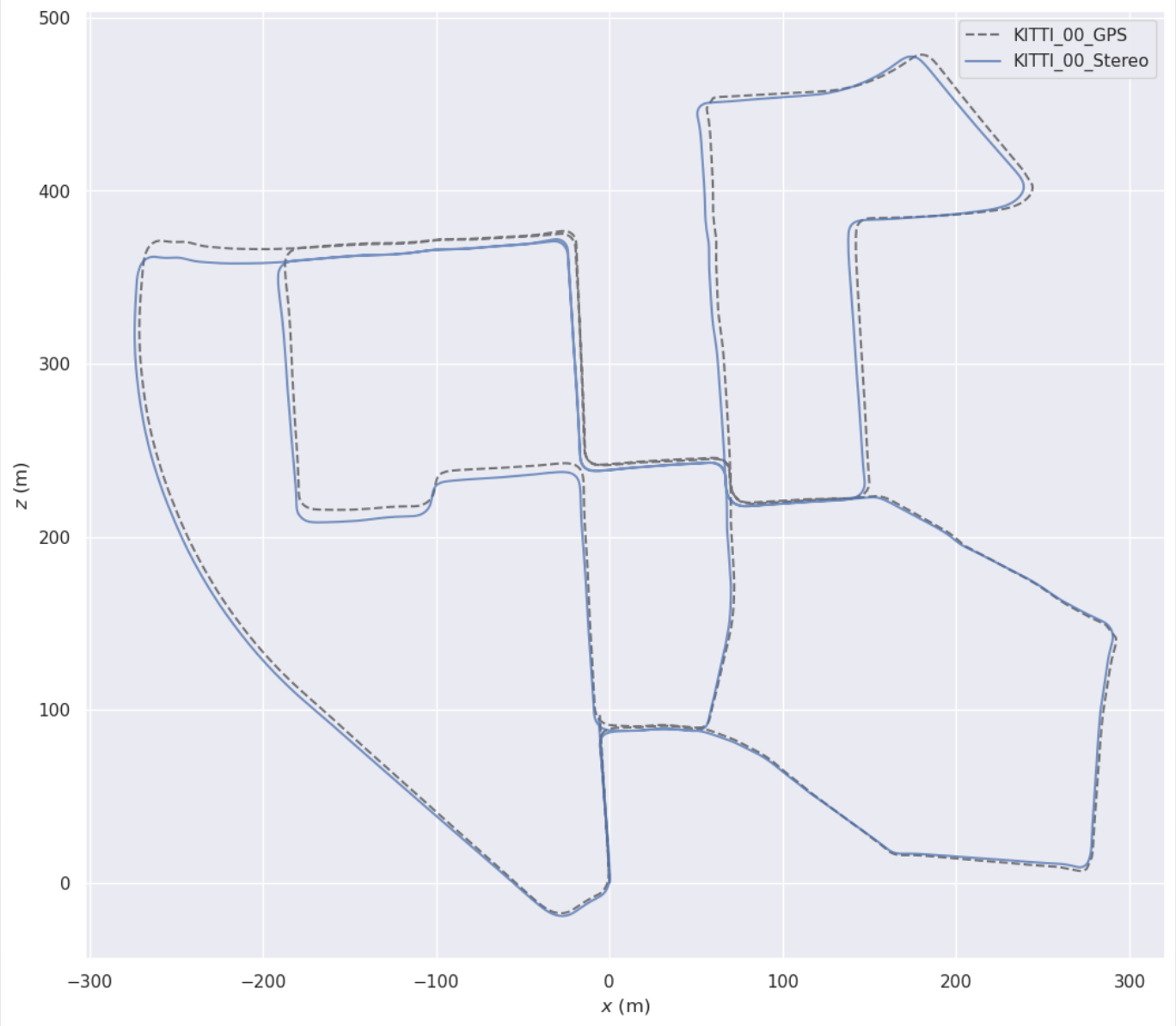
- *Evaluating the Quality of the Joint State Estimate*
- Estimating the Quality of the Platform Estimate
- *Estimating the Robustness of the Map*
- *Estimating the Localization Ability of the Map*

Estimating the Quality of the Platform Estimate

- Since evaluating the positions of the landmarks is a bit fraught, we can focus on just the *quality of the platform estimate:*
  - It's easy to work out mathematically
  - It's what we are interested in anyway
- We'll demonstrate this using the KITTI-00 dataset
- This has ground truth provided by a separate navigation system on the vehicle


00
500
400
300
200
100
0
z (m)
−200
−100
0
100
200
300
x (m)


KITTI_00_Stereo
500
400
300
200
100
0
z (m)
−300
−200
−100
0
100
200
300
x (m)

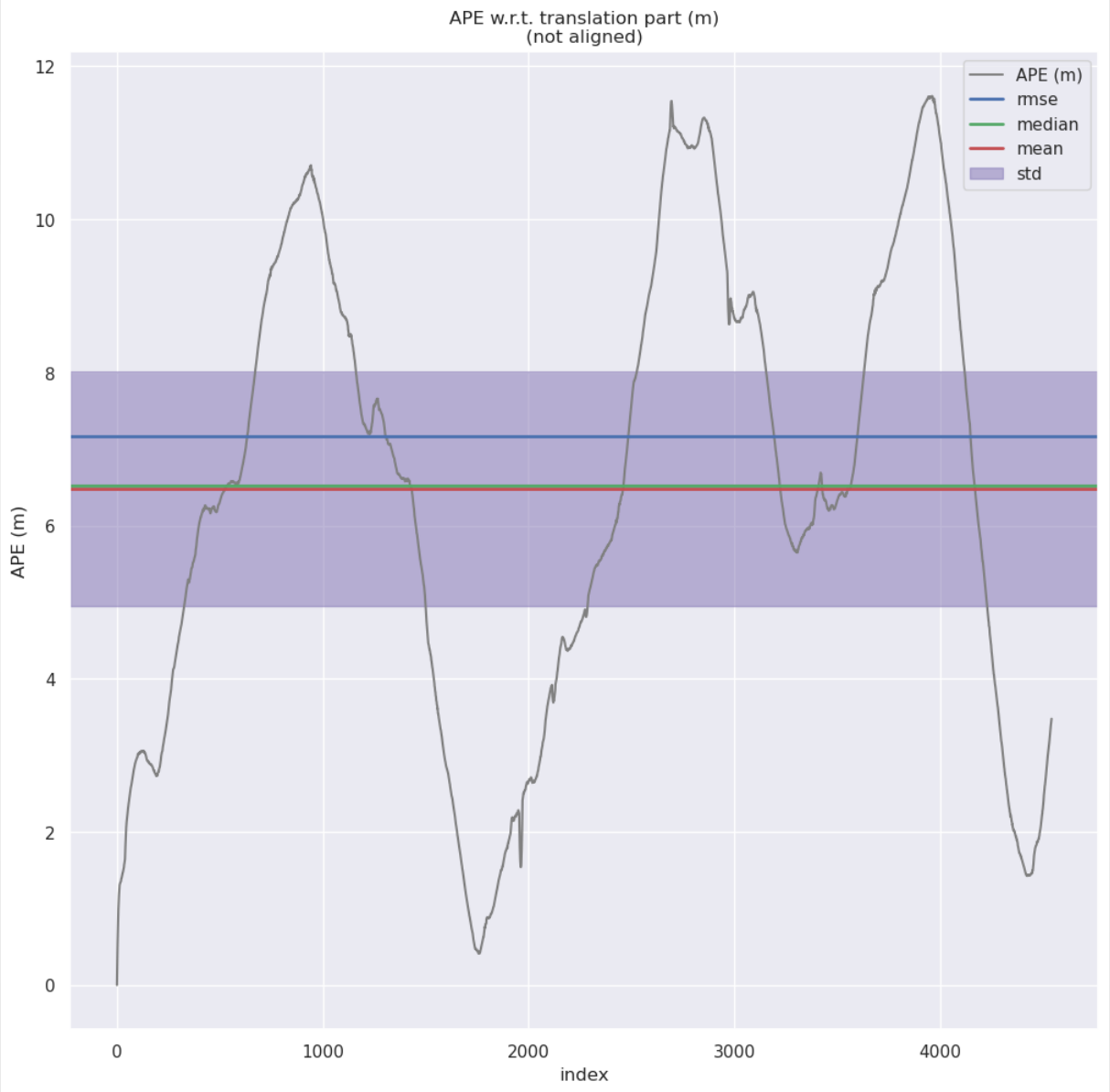

KITTI_00_GPS
KITTI_00_Stereo
500
400
300
200
100
0
z (m)
−300
−200
−100
0
100
200
300
x (m)

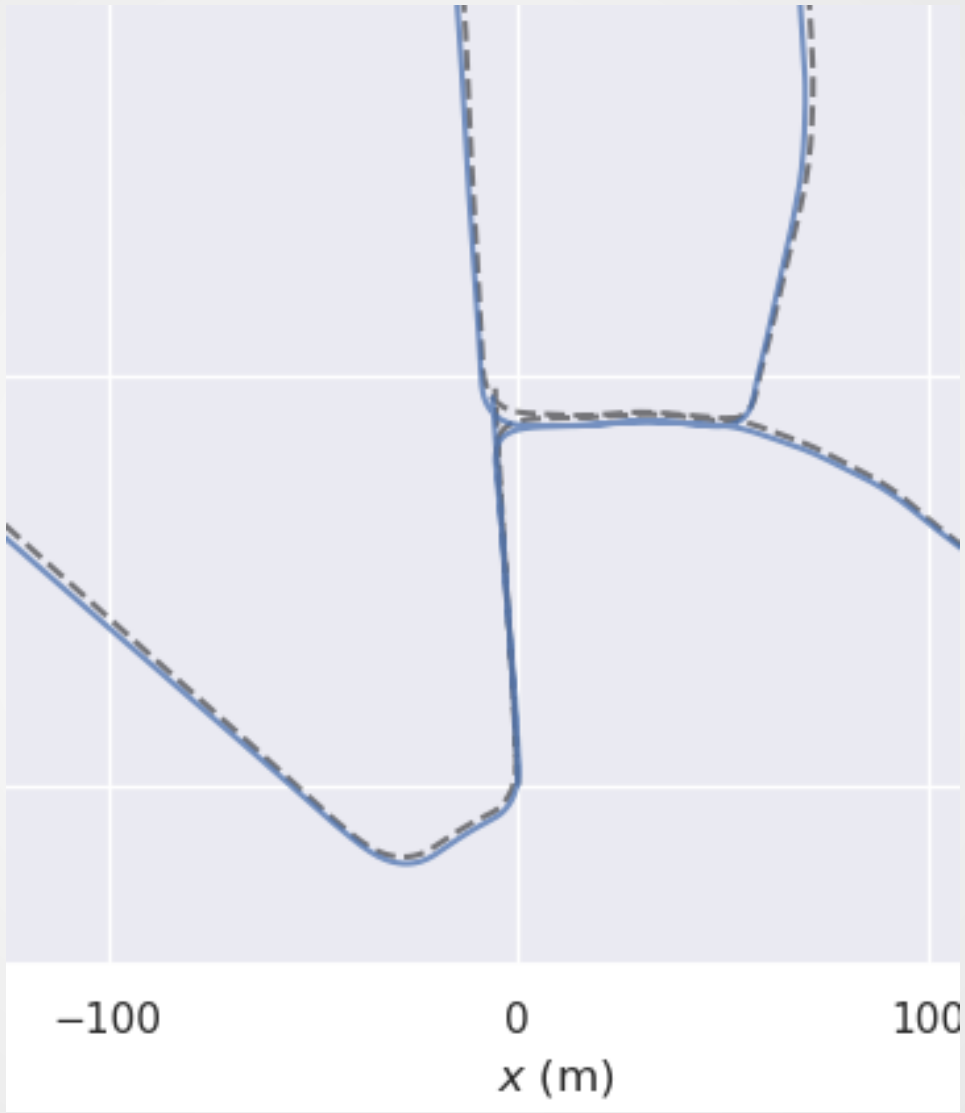
Absolute Trajectory Error / Absolute Pose Error

- This is given by the difference between the estimated and ground truth platform position
- We define an SE(n) error using our box minus operator

$$\tilde{\mathbf{e}}_k = \mathbf{x}_k \boxminus \hat{\mathbf{x}}_k$$

- This is the same as the earlier case, but we use SE(3) because this is visual SLAM


APE w.r.t. translation part (m)
(not aligned)
APE (m)
rmse
median
mean
std
APE (m)
index
0
2
4
6
8
10
12
0
1000
2000
3000
4000


−100
0
100
x (m)

Misalignment from Initial Conditions

- One reason for the misalignment between the two is that the initial conditions in ORB-SLAM are *arbitrary*
- The initial position and orientation are both set to 0
- However, the ground truth uses GNSS which does not apply this offset
- The initial condition error doesn't tell us anything really interesting about the performance of the SLAM system

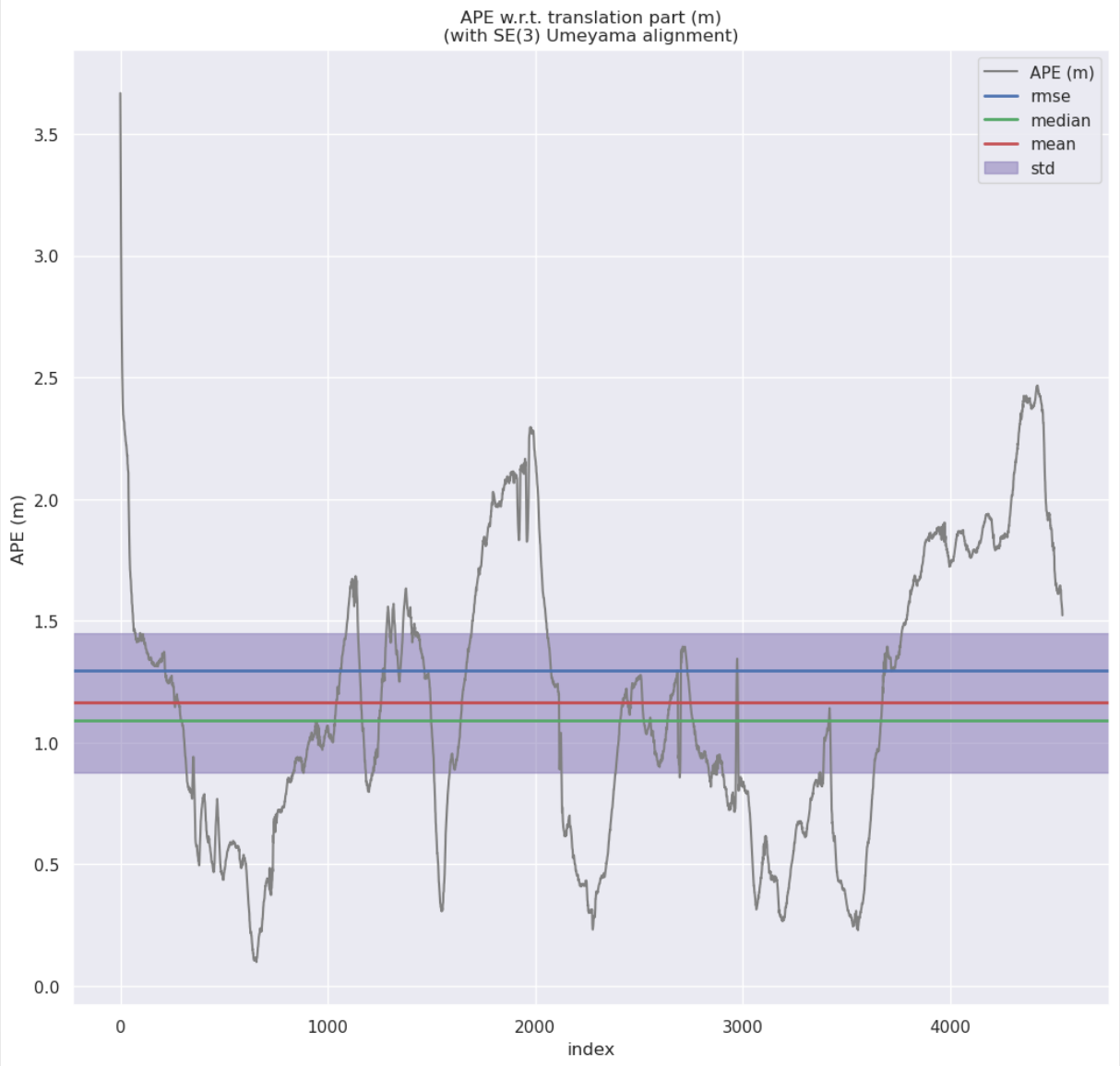
Aligning Trajectories

- Therefore, a common approach is to align the estimated and ground truth trajectories as closely as possible and then compute the error
- The alignment is assumed to be a rigid transformation,

$$\mathbf{x}_k = \mathbf{R}\hat{\mathbf{x}}_k + \mathbf{t}$$

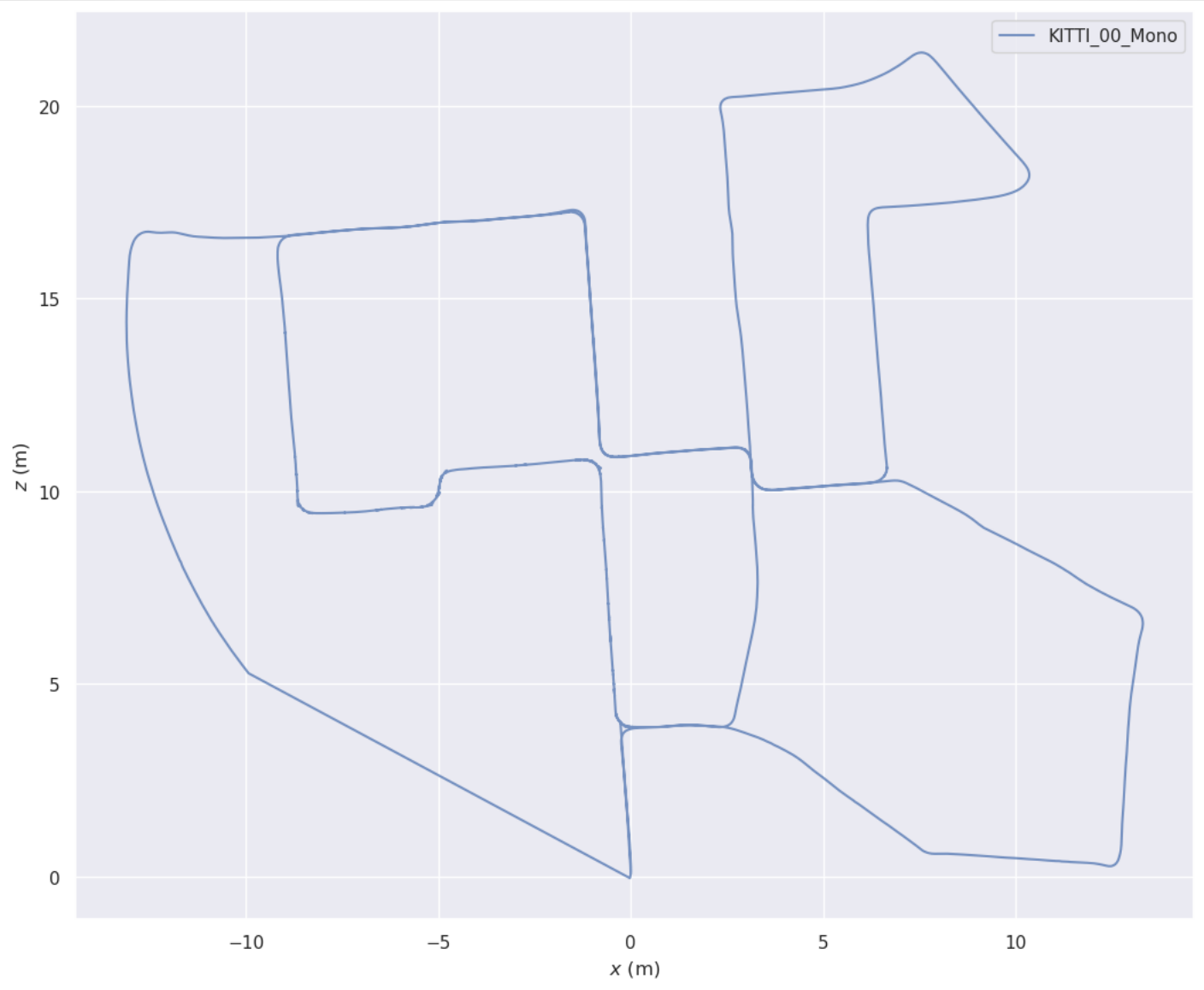
- Umeyama Alignment is widely used
- This is based on SVDs


KITTI_00_GPS
KITTI_00_Stereo
500
400
300
200
100
0
z (m)
−200
−100
0
100
200
300
x (m)


APE w.r.t. translation part (m)
(with SE(3) Umeyama alignment)
APE (m)
rmse
median
mean
std
APE (m)
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
1000
2000
3000
4000
index

Alignment and Scale Drift

- Things become more difficult with monocular cameras
- Because monocular cameras can’t measure depth, the scale is fixed to an arbitrary value

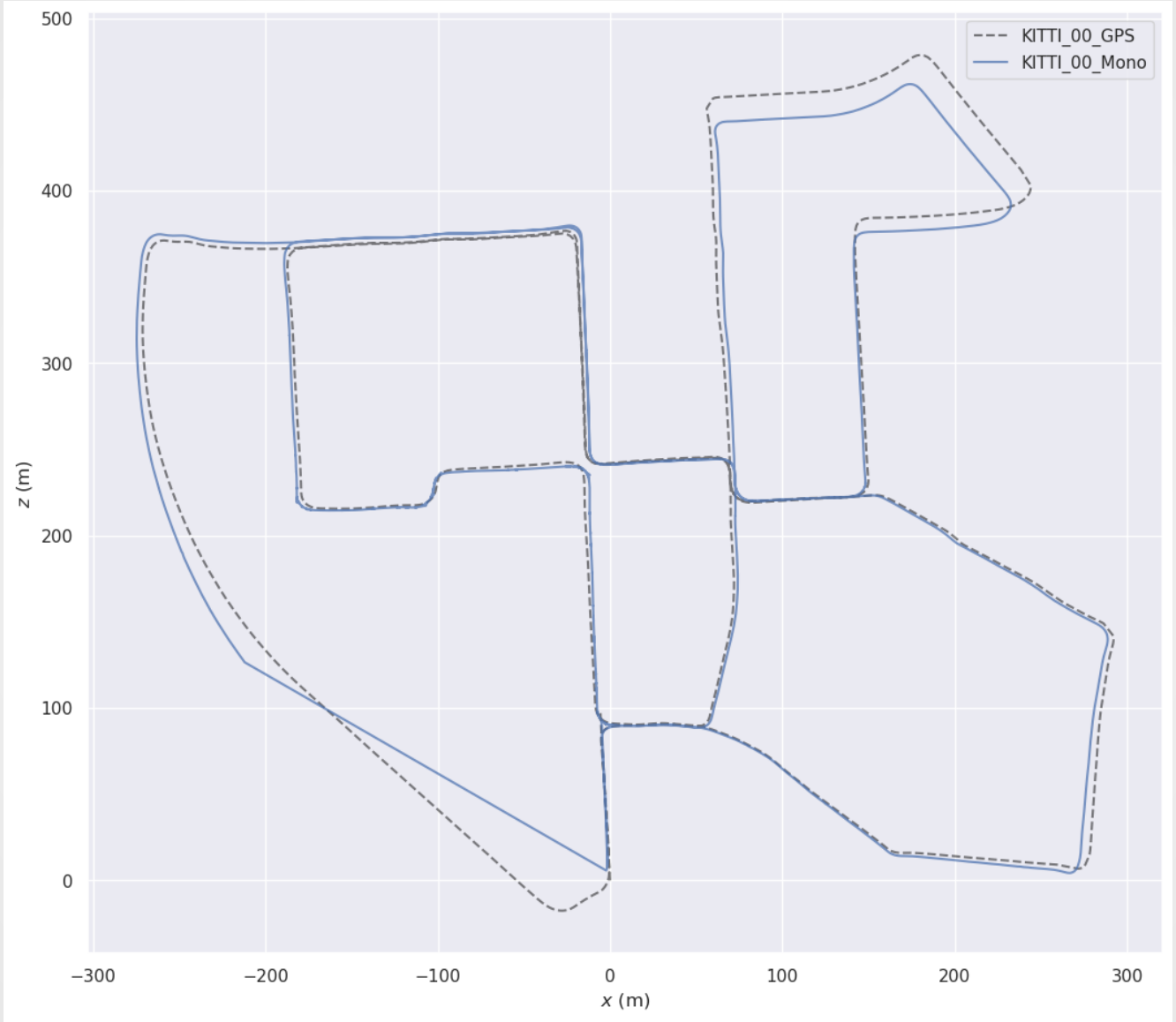

KITTI_00_Mono
20
15
10
5
0
z (m)
−10
−5
0
5
10
x (m)


KITTI_00_GPS
KITTI_00_Mono
500
400
300
200
100
0
z (m)
−200
−100
0
100
200
300
x (m)

Alignment and Scale Drift

- Things become more important with monocular cameras
- Because monocular cameras can't measure depth, the scale is fixed to an arbitrary value
- The alignment between the estimated and ground truth is a seven-dimensional transformation,

$$\mathbf{x}_k = s\mathbf{R}\hat{\mathbf{x}}_k + \mathbf{t}$$

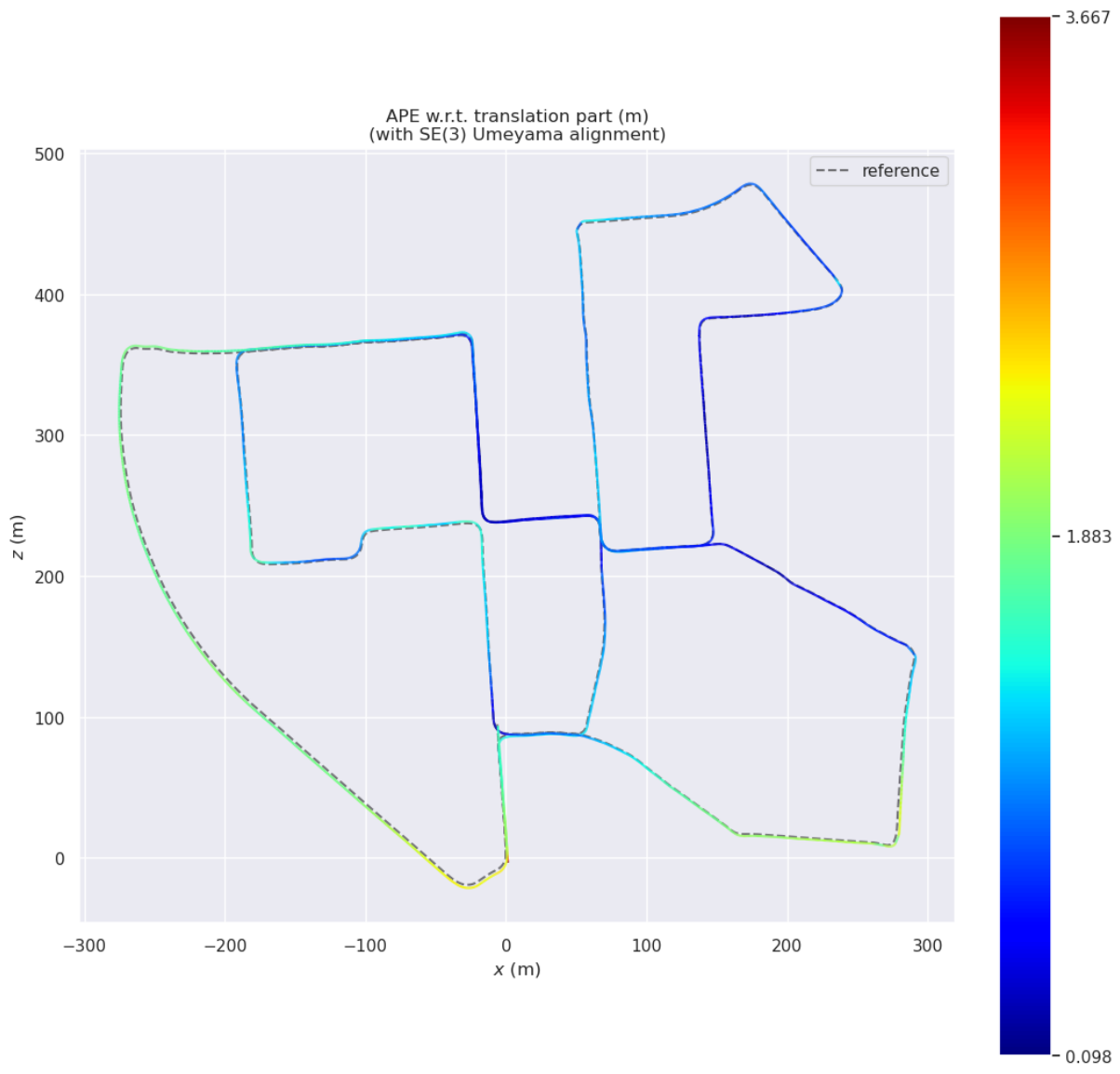

KITTI_00_GPS
KITTI_00_Mono
500
400
300
200
100
0
z (m)
−300
−200
−100
0
100
200
300
x (m)


KITTI_00_Stereo
KITTI_00_Mono
500
400
300
200
100
0
z (m)
−300
−200
−100
0
100
200
300
x (m)

Limitations of Absolute Trajectory Error

- Aligned estimates work well when the performance of the SLAM system is good everywhere
- However, local deformation can cause poor solutions with Umeyama
- It’s not clear whether the error growth is systemic or gradual


KITTI_00_Stereo
KITTI_00_Mono
500
400
300
200
100
0
z (m)
−300
−200
−100
0
100
200
300
x (m)


APE w.r.t. translation part (m)
(with SE(3) Umeyama alignment)
reference
500
400
300
200
100
0
z (m)
−300
−200
−100
0
100
200
300
x (m)
3.667
1.883
0.098

Relative Pose Error

- A way to address this is to consider the relative pose error
- This is the error in the pairwise changes in pose
- It is local
- It does not rely on global alignment or scale estimates

Relative Pose Error

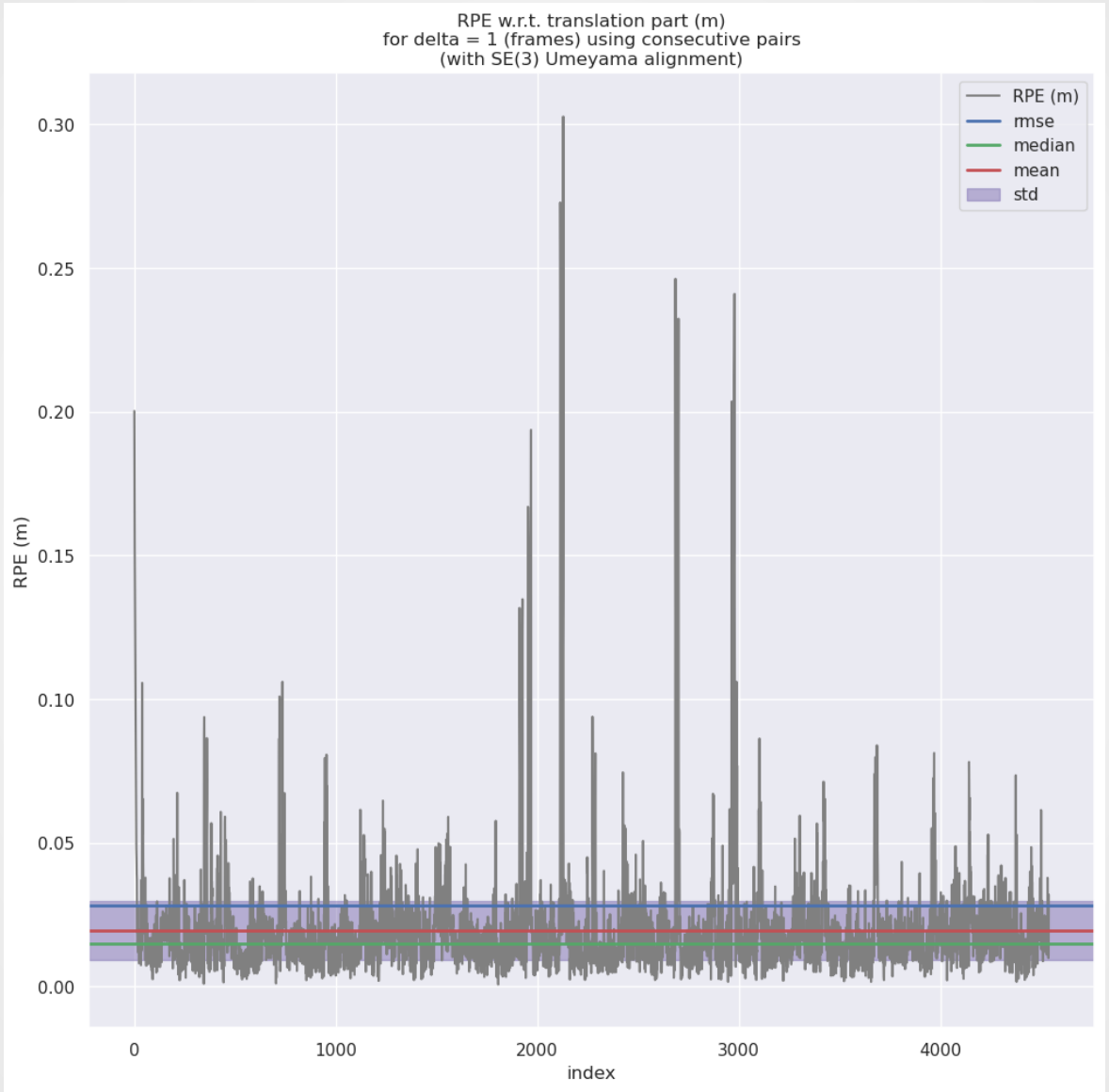
- Compute the change for both the estimate and ground truth,

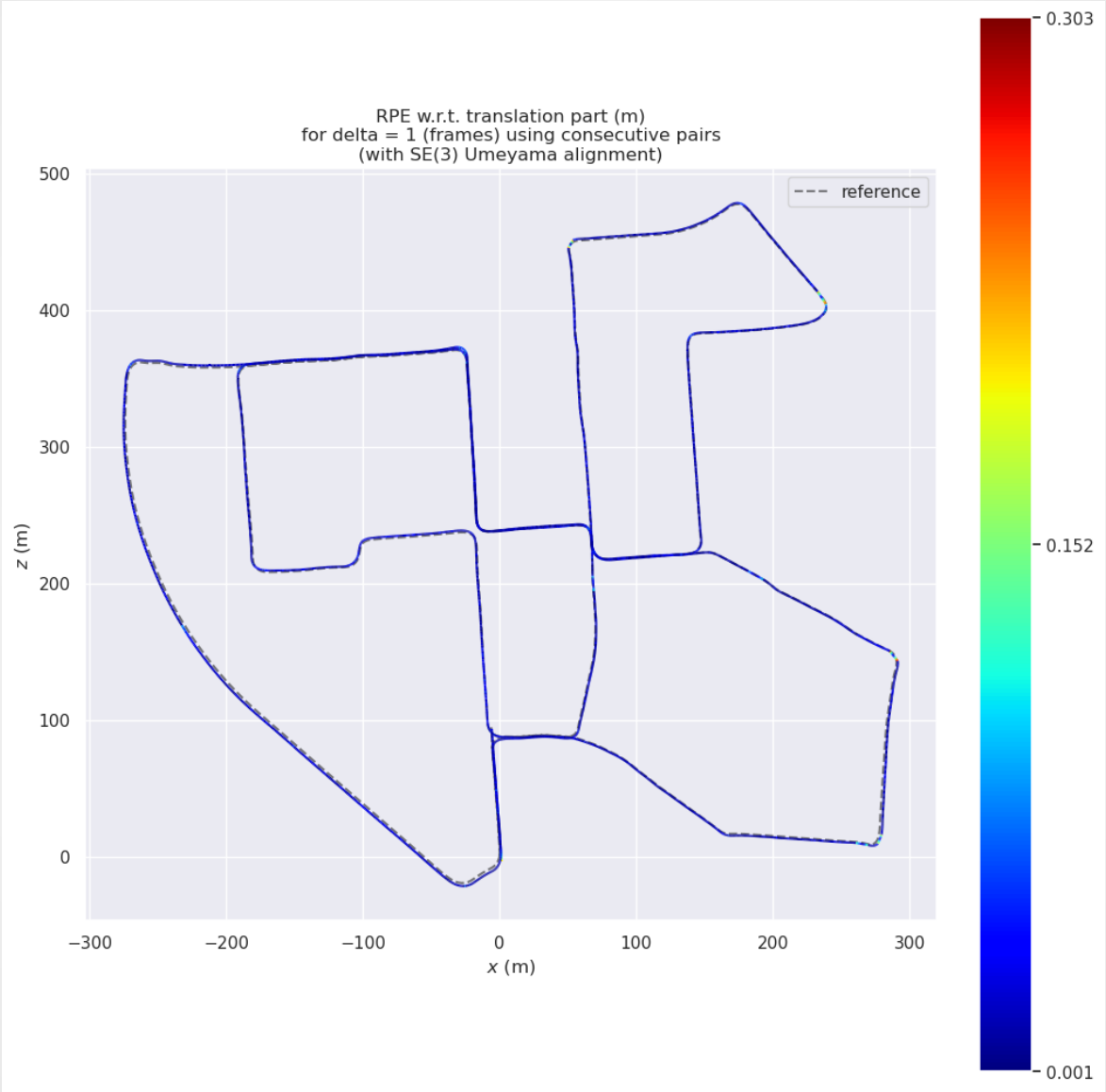
$$\mathbf{R}_{\mathbf{est}}\left[k\right] = \mathbf{R}^{-1}\left[\hat{\mathbf{x}}_{k-1}\right]\mathbf{R}\left[\hat{\mathbf{x}}_{k}\right]$$

$$\mathbf{R}_{\mathbf{true}}\left[k\right] = \mathbf{R}^{-1}\left[\mathbf{x}_{k-1}\right]\mathbf{R}\left[\mathbf{x}_{k}\right]$$

- The RPE is then

$$\mathbf{RPE}\left[k\right] = \mathbf{R}_{\mathbf{true}}^{-1}\left[k\right]\mathbf{R}_{\mathbf{est}}\left[k\right]$$


RPE w.r.t. translation part (m)
for delta = 1 (frames) using consecutive pairs
(with SE(3) Umeyama alignment)
RPE (m)
rmse
median
mean
std
0.30
0.25
0.20
0.15
0.10
0.05
0.00
RPE (m)
0
1000
2000
3000
4000
index


RPE w.r.t. translation part (m)
for delta = 1 (frames) using consecutive pairs
(with SE(3) Umeyama alignment)
reference
500
400
300
200
100
0
z (m)
−300
−200
−100
0
100
200
300
x (m)
0.303
0.152
0.001

Summary

- We can estimate the pose of the platform
- However, this just tells us how well the robot performed *in one particular case*
- It doesn't tell us any general information about how robust the solution or or how good the map is
- We'll next look at robustness techniques

Estimating the Robustness of the Map

- *Evaluating the Quality of the Joint State Estimate*
- *Estimating the Quality of the Platform Estimate*
- Estimating the Robustness of the Map
- *Estimating the Localization Ability of the Map*

Estimating the Robustness of the Map

- We'd like to get a sense of how repeatable performance is
- However, there are many factors which influence a run, including:
  - Stochastic processes in feature extraction
  - Stochastic processes in RANSAC
- Therefore, the easiest way to assess this is through sampling

SiTAR

- From the well-known novel (Anna Karenina):
  - All happy families are alike; each unhappy family is unhappy in its own way.

- SiTAR snappier version:
  - All happy tracking systems produce similar trajectories; each unhappy tracking system generates an unhappy track in its own unhappy way

SiTAR Philosophy

SiTAR Computation

- Run the *same* data set through the *same* tracking system multiple times
- Record the generated trajectories for each run

$$\mathbf{T}^{(i)} = \left\{ \hat{\mathbf{x}}_0^{(i)}, \cdots, \hat{\mathbf{x}}_N^{(i)} \right\}$$

SiTAR Computation

- **Within each run compute the δ relative transformations,**

$$\mathbf{R}_{\mathbf{est}}^{(i)}[k] = \mathbf{R}^{-1}\left[\hat{\mathbf{x}}_{k-\delta}^{(i)}\right]\mathbf{R}\left[\hat{\mathbf{x}}_{k}^{(i)}\right]$$

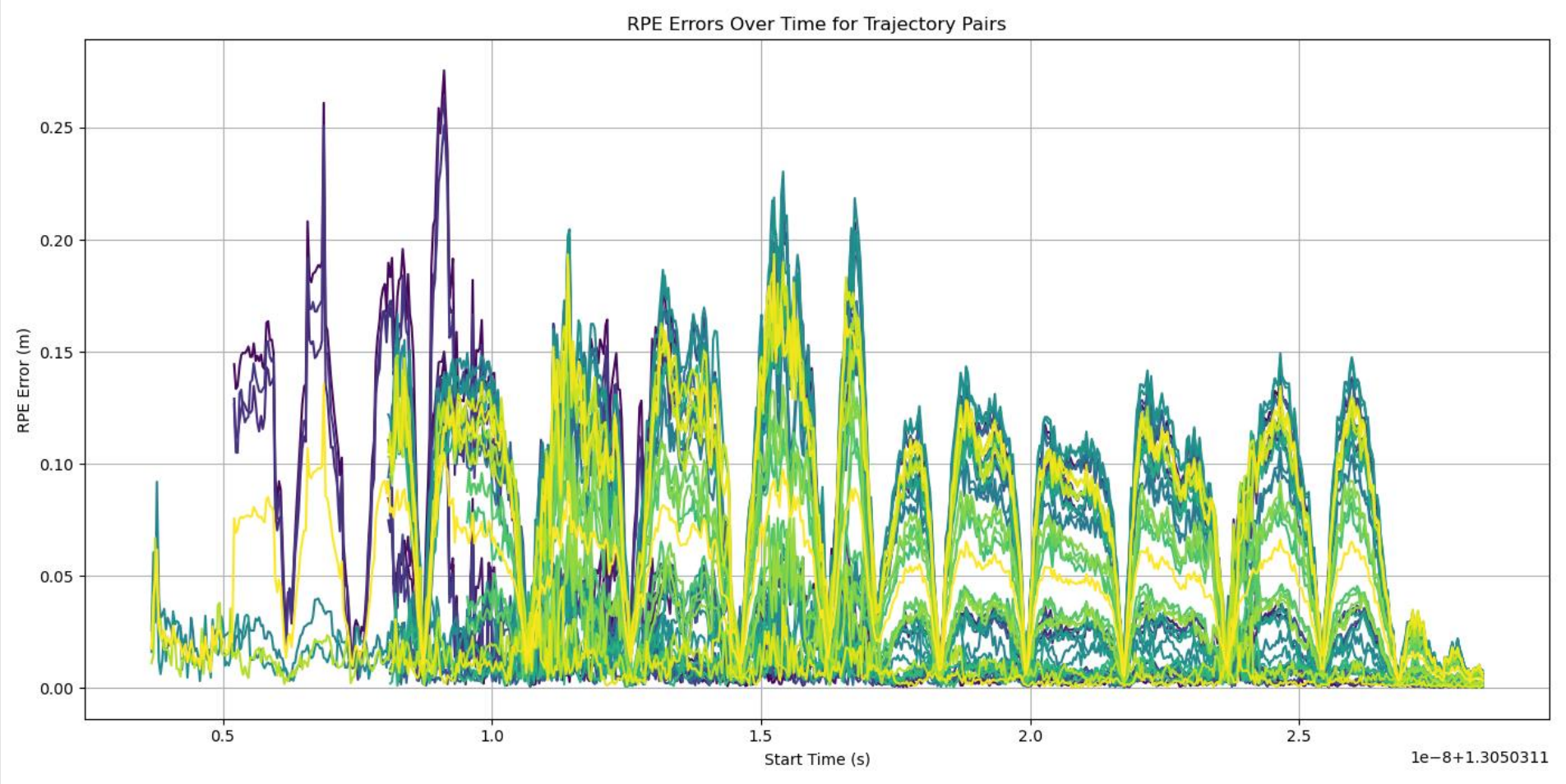
- **Finally, compute the RPE between all pairs of runs,**

$$\mathbf{RPE}_{\mathbf{est}}^{(i,j)}[k] = \mathbf{RPE}_{\mathbf{est}}^{(i)}[k]\left(\mathbf{RPE}_{\mathbf{est}}^{(j)}[k]\right)^{-1}$$

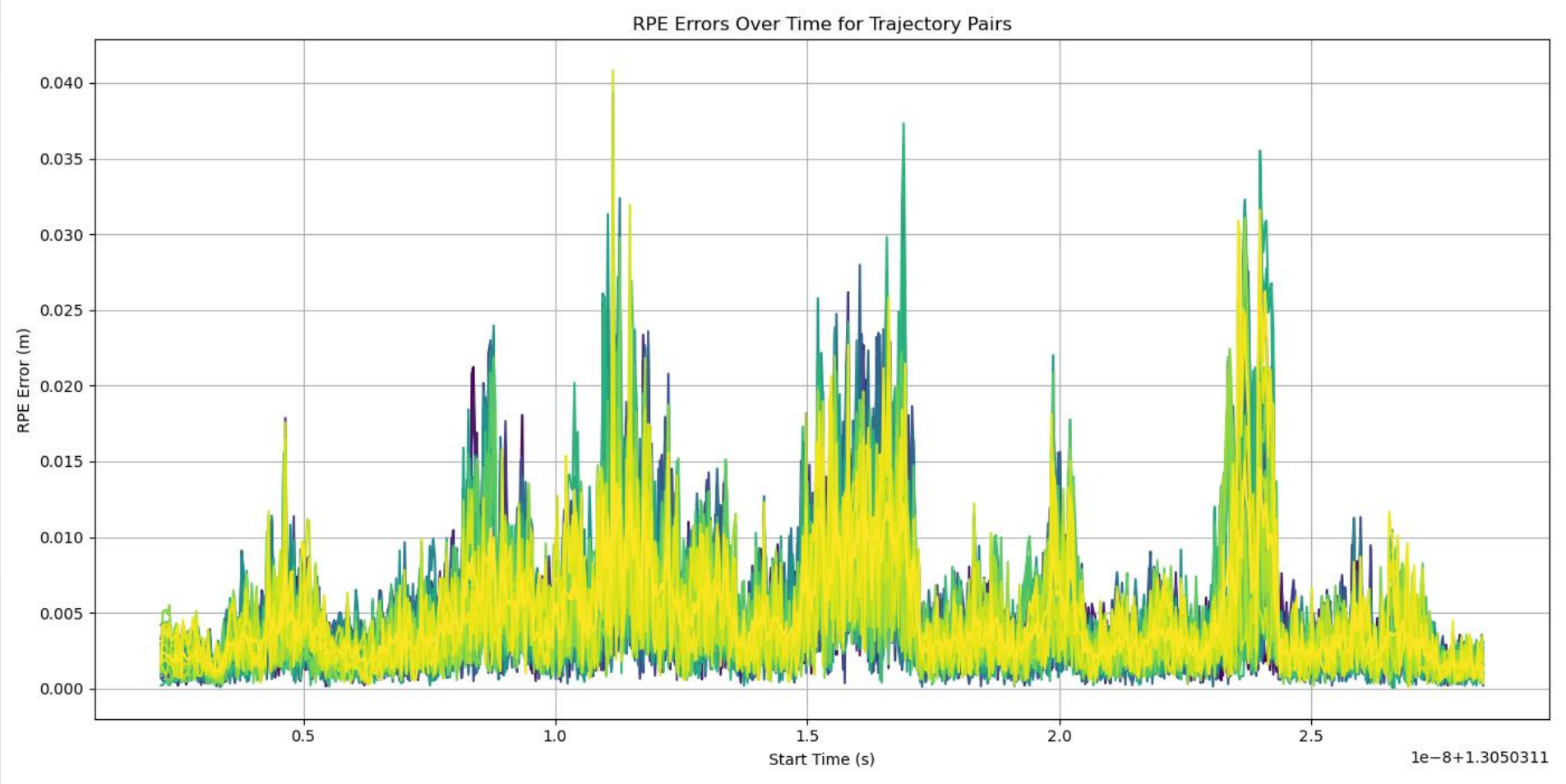
TUM FR1 XYZ Dataset Run 1

TUM FR1 XYZ Dataset Run 2

SiTAR Mono Results from 10 Runs

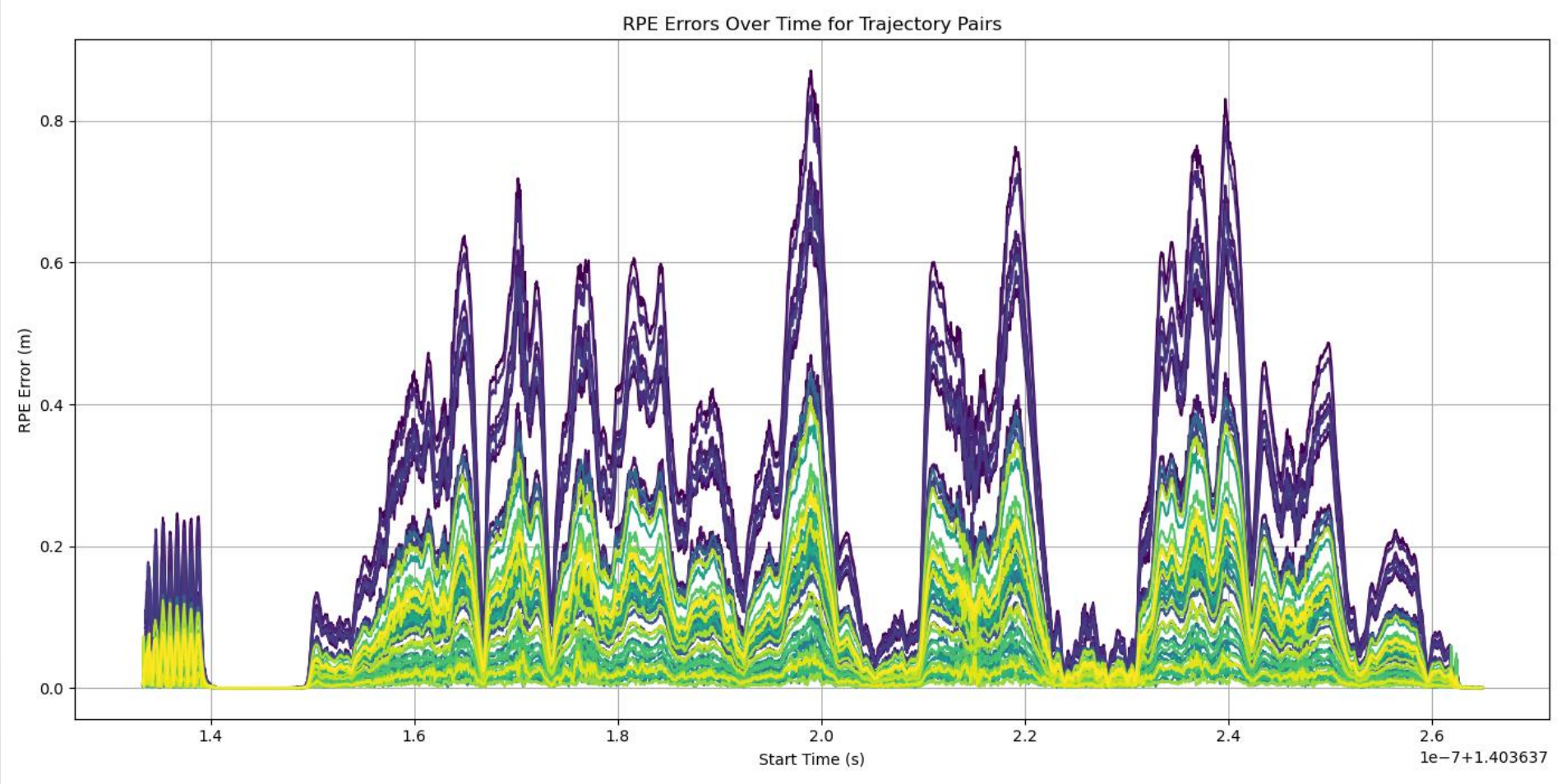


SiTAR RGB-D Results from 10 Runs

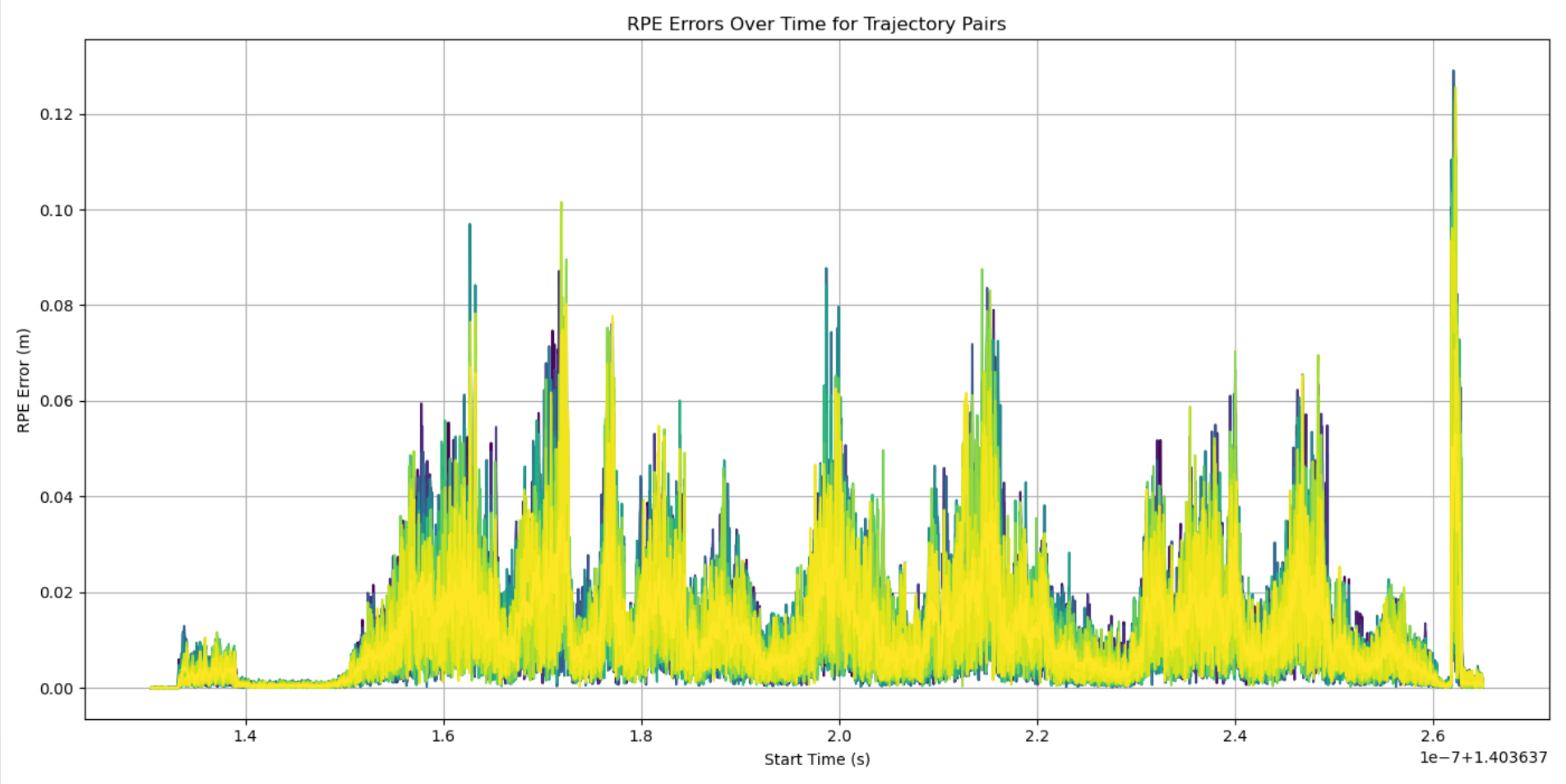


EuRoC Machine Hall 3 Stereo Dataset

SiTAR Mono Results from 10 Runs



SiTAR Stereo Results from 10 Runs



Randomized Variant

- Recently, another set of authors came up with a similar approach to SiTAR but added an extra refinement
- In addition to running the same dataset through multiple times, they generate additional datasets by adding noise to the frames
- They compute APE by doing a mix of:
  - Using the same data over and over again
  - Randomising the data by corrupting it with additive Gaussian noise

Summary

- Sampling based techniques can let us explore how robust a particular platform trajectory is
- However, we'd like to predict how good the map will be for any trajectory
- Therefore, we'll finish up with methods which try to predict the localization performance from the map

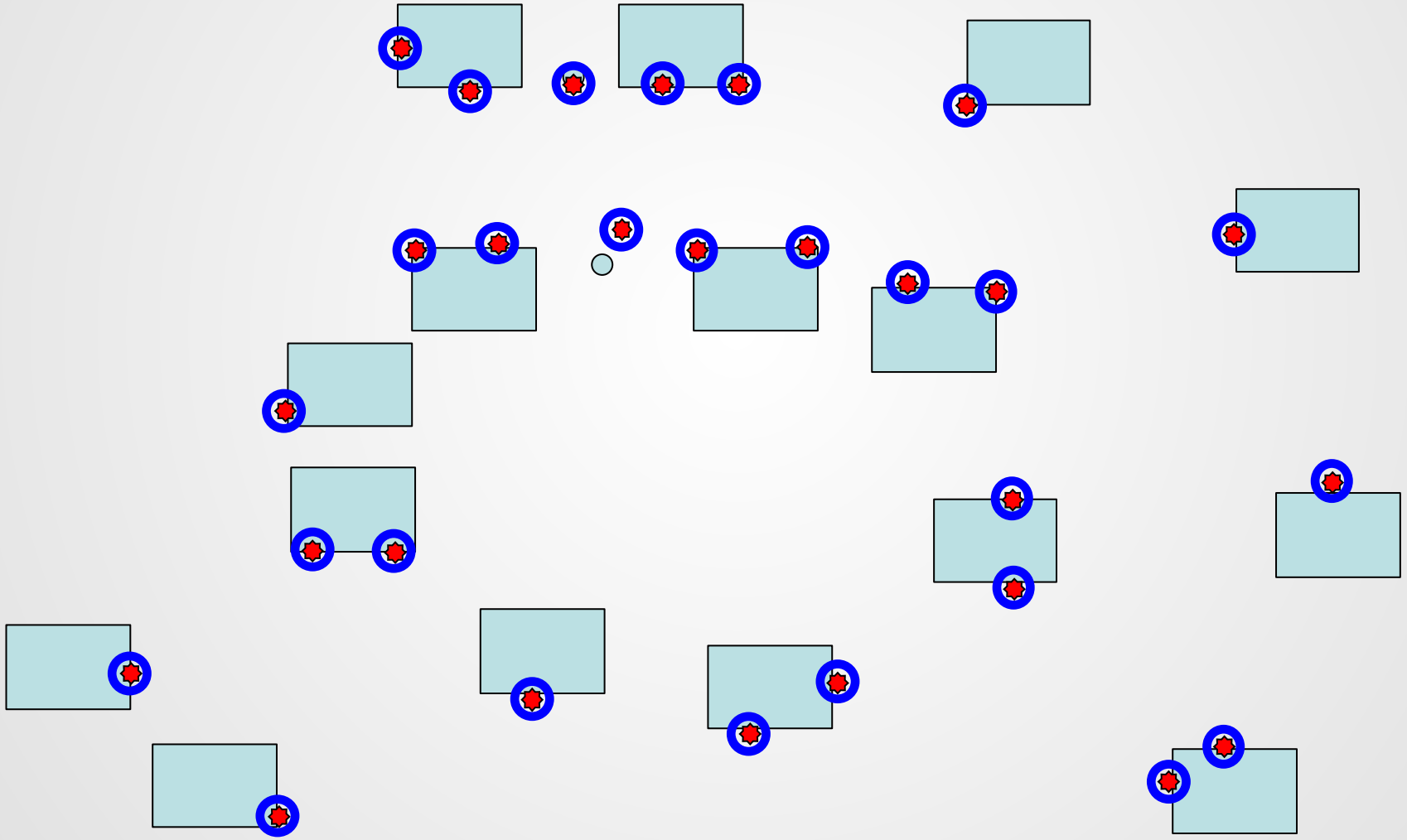
Estimating the Localization Ability of the Map

- *Evaluating the Quality of the Joint State Estimate*
- *Estimating the Quality of the Platform Estimate*
- *Estimating the Robustness of the Map*
- Estimating the Localization Ability of the Map

Map Quality Evaluation for Platform Localization

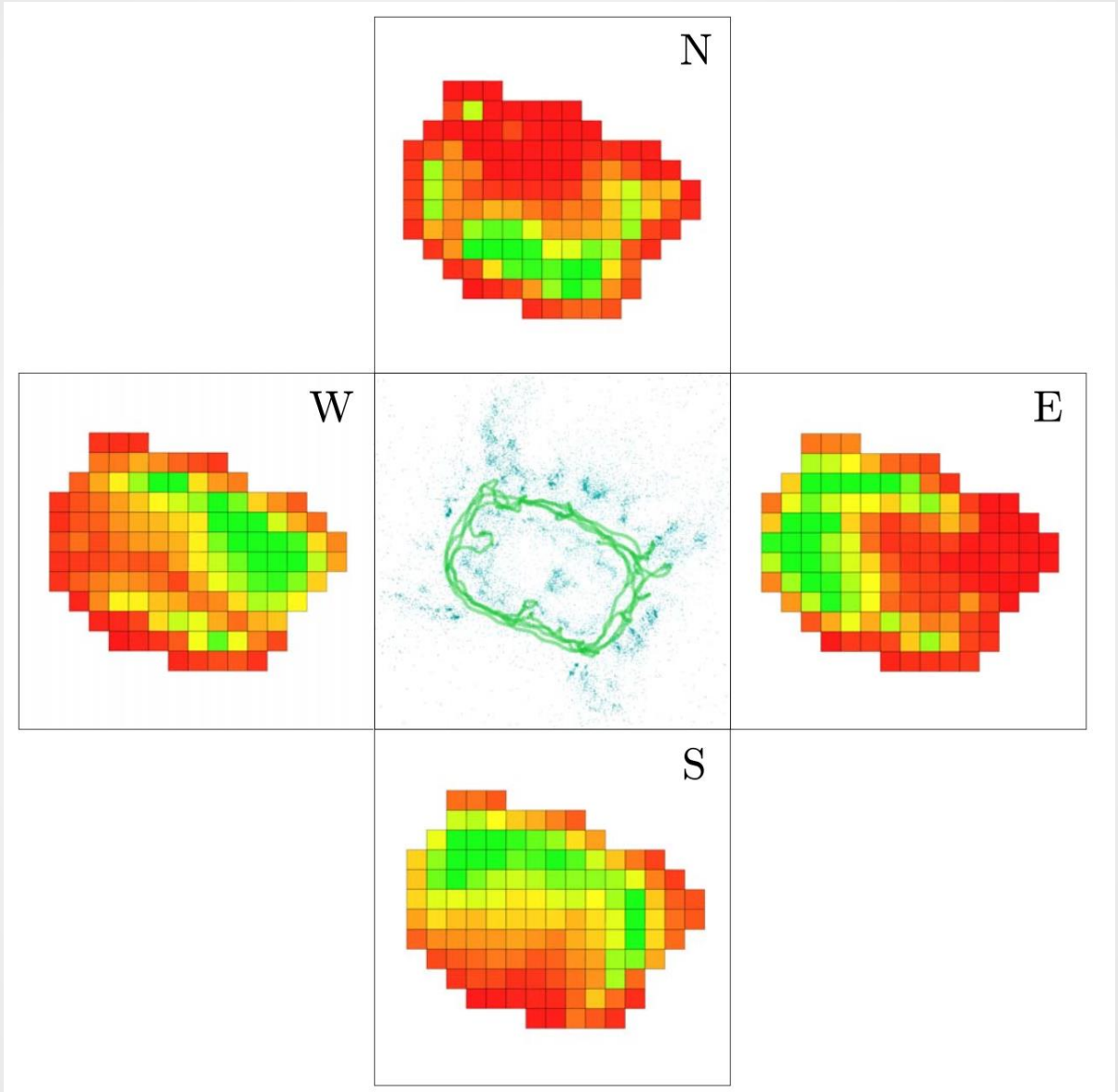
- The idea is that, given a map, can you predict how good the map will be for localizing an arbitrary trajectory of a camera

Map Quality Evaluation for Platform Localization



Map Quality Evaluation for Platform Localization

- The idea is that, given a map, can you predict how good the map will be for localizing an arbitrary trajectory of a camera
- One way to do this is to use a geometric technique and dense sampling of camera locations



"Map Quality Evaluation for Visual Localization", Merzic et al., 2017

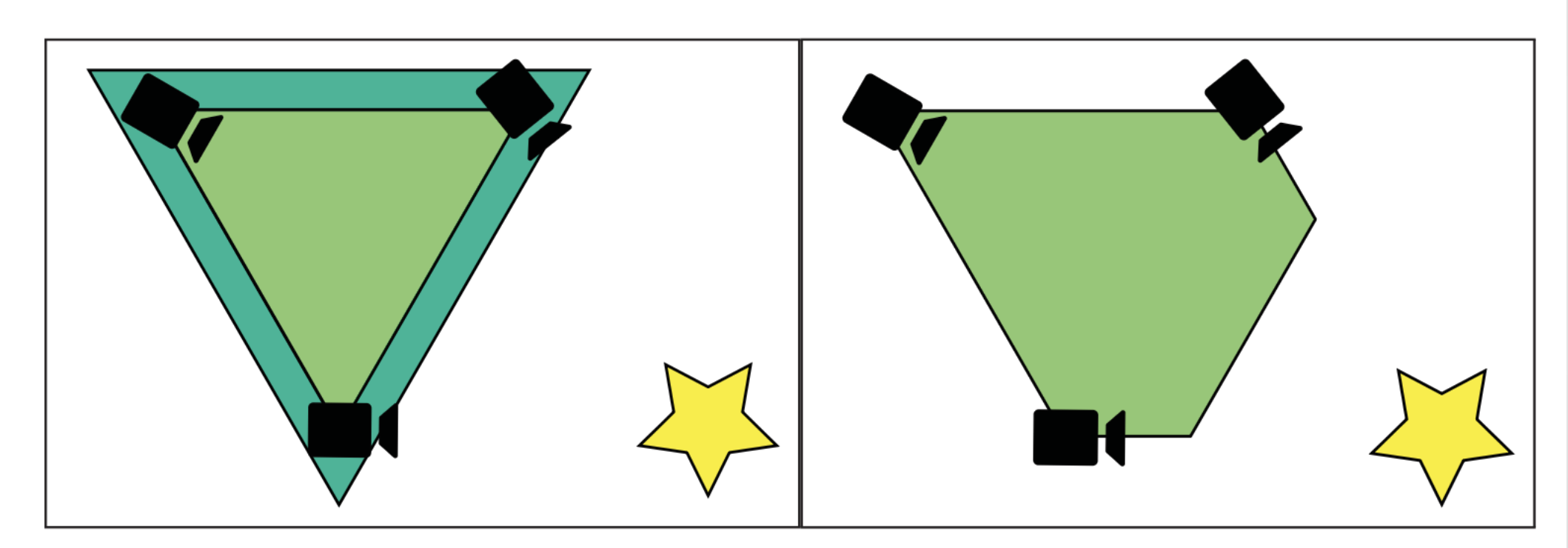
Approach

- Specify a trajectory you are interested in as a set of platform poses
- From each pose, figure out which landmarks are visible
- Score each visible landmark
- Fuse the landmark scores together and normalize

Figuring out Landmark Visibility

- The Co-visibility Graph in ORB-SLAM2 links keyframes together but doesn’t show the complete space where a landmark can be seen
- Method approximates using the convex hull of all the keyframe positions where a landmark is visible from

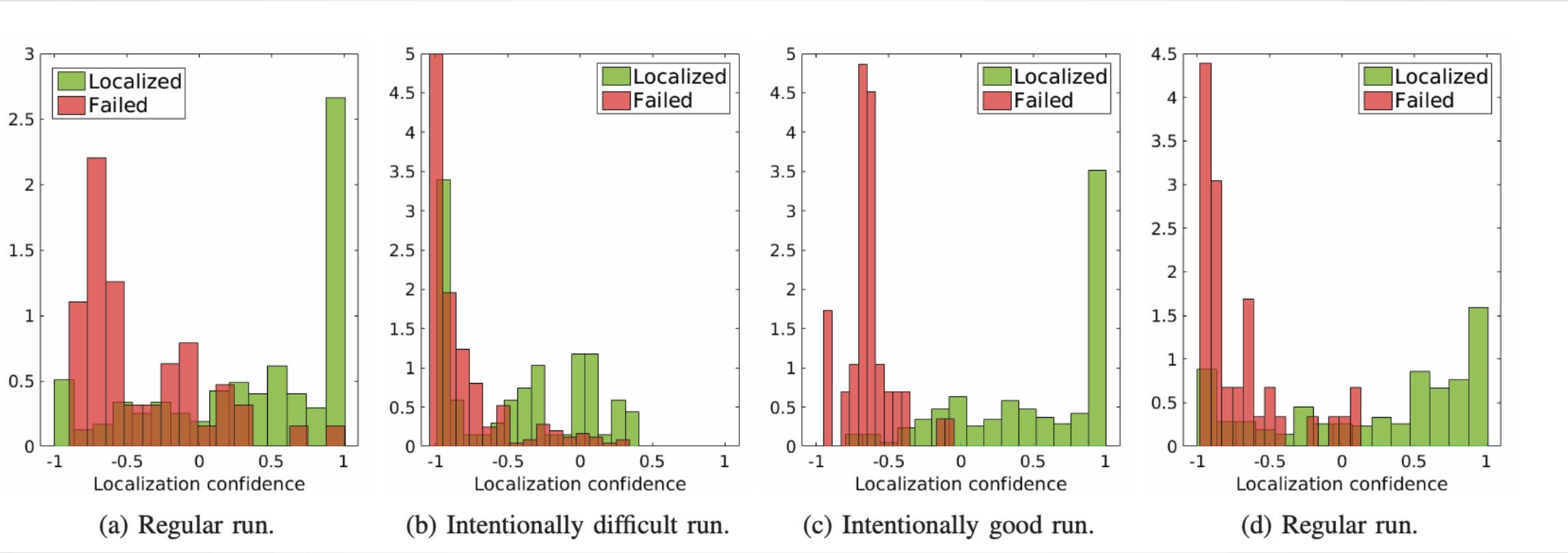
Convex Hulls for Visibility



Convex Hull Scoring and Fusion

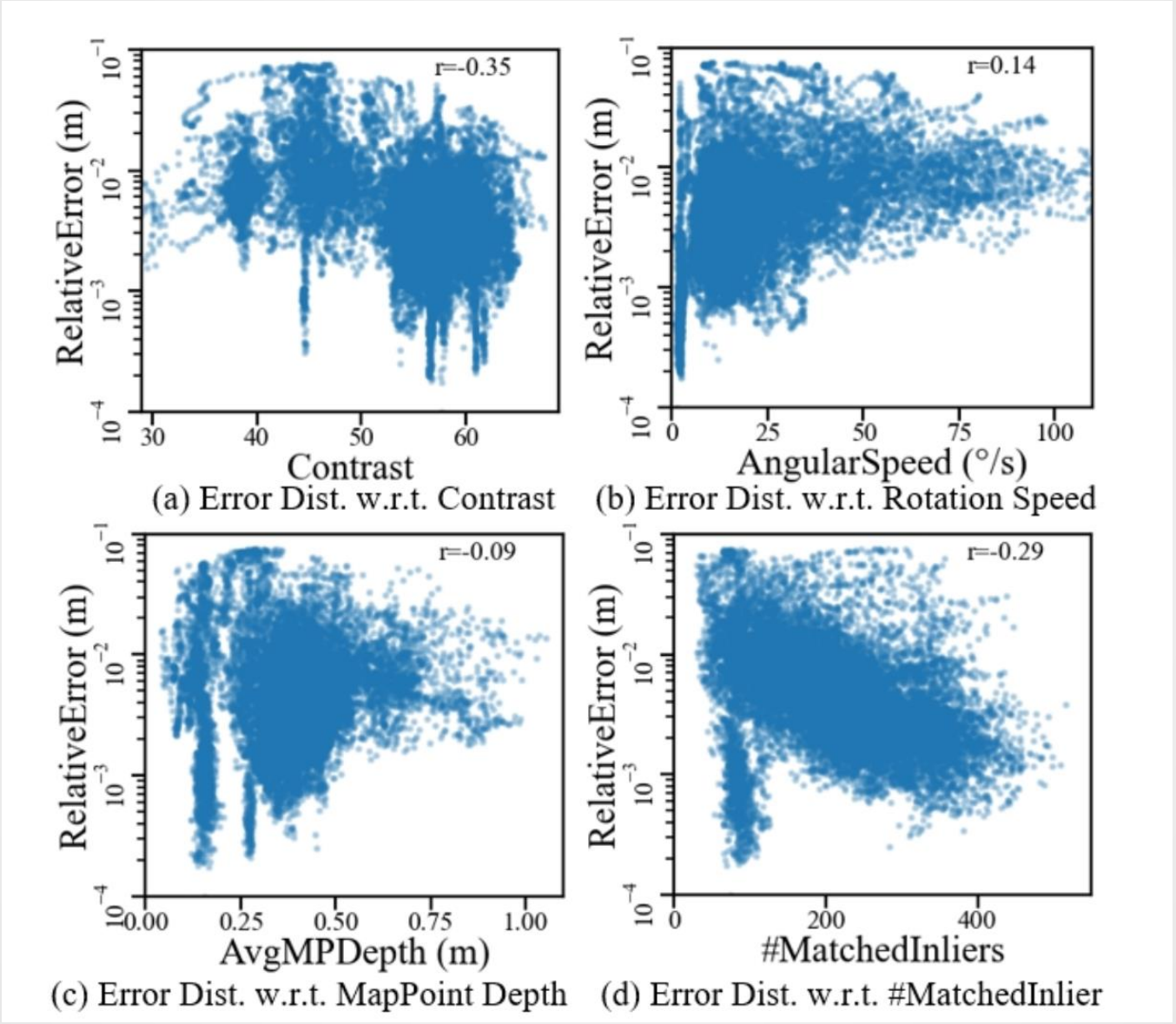
- Each landmark is scored based on the number of observers visible
- The scores from all the hulls are added together
- The results are normalized in the range [-1, 1]:
  - -1 means “never localizes”
  - 0 means “localizes sometimes”
  - 1 means “best localization performance”

Results


Localized
Failed
Localization confidence
(a) Regular run.
(b) Intentionally difficult run.
(c) Intentionally good run.
(d) Regular run.

Acccuracy Regression

- Another approach is to regress directly from the input images and other contextual variables



(a) Error Dist. w.r.t. Contrast (b) Error Dist. w.r.t. Rotation Speed

(c) Error Dist. w.r.t. MapPoint Depth (d) Error Dist. w.r.t. #MatchedInlier

Acccuracy Regression

- Another approach is to regress directly from the input images and other contextual variables
- A network (DeepSEE) is trained to regress tracker accuracy from the provided data
- The training data includes simulations and ground truth data from a motion capture system

Summary

- Another way to evaluate the map is the quality of the computed pose from the map

- This uses geometric information or other contextual information to predict sensing system performance

Umeyama Alignment

- This uses SVDs (!) to compute the alignment